FINAL PROJECT
SPECIFICATION

QoS Traffic Manager

Packet Prioritization and Performance Evaluation under Network Congestion

PROJECT IN ONE SENTENCE

A lightweight software simulation that generates four classes of network traffic, compares FIFO with Strict Priority Queuing on a single congested link, and presents measured delay, loss, throughput, queue, and utilization results in an interactive dashboard.

Project at a glance

Item	Final decision
Project type	Computer Networks CBP; simulated traffic and packet queuing
Core question	How does priority scheduling change service for different traffic classes when demand exceeds link capacity?
Traffic classes	Voice, video, web, and bulk file transfer
Policies compared	FIFO baseline versus Strict Priority Queuing (SPQ)
Network model	One bottleneck link, one finite shared buffer, fixed propagation delay
Results	Per-class and aggregate metrics, comparison charts, packet-event trace, CSV export
Scope level	Small, demonstrable, and bounded; not a production network simulator

Document status: Final scope baseline | Version: 1.0 | Date: 12 October 2026

SCOPE RULE

The project models the core effect of traffic classification and packet scheduling. It does not claim to implement every mechanism in a complete QoS architecture or reproduce a real Internet path.

1. Problem statement

A network link has finite transmission capacity. Voice, video, web, and file-transfer traffic may compete for that same link. When their combined offered load exceeds available capacity, packets wait in queues; if the buffer becomes full, new packets may be dropped. Applications do not react equally to delay: interactive voice and video are generally more delay-sensitive than bulk file transfers.

The project will model this situation and compare an ordinary arrival-order scheduler (FIFO) with a priority-based scheduler (SPQ). The comparison will use the same generated packet workload and identical network settings for both policies.

2. Objectives

- Represent four application traffic classes with different priorities and representative packet-size/arrival patterns.
- Model a single constrained link and a finite packet buffer.
- Implement FIFO as the baseline service policy and Strict Priority Queuing as the QoS policy.
- Introduce low-load, near-capacity, and overloaded scenarios.
- Calculate packet delay, delay variation, packet drops, delivered throughput, queue occupancy, and link utilization.
- Show aggregate and per-class results in an interactive, presentation-ready dashboard.
- Make runs reproducible so the comparison can be repeated and verified.

3. What “QoS” means in this project

Quality of Service (QoS) is a broad set of network mechanisms for differentiating and managing traffic. This project implements only a small, clearly defined subset: logical traffic classification, priority assignment, queueing, packet scheduling, finite-buffer drop tracking, and performance measurement. It does not implement a complete Differentiated Services (DiffServ) network.

4. System boundary

Included in the model	Outside the model
Synthetic packet generation for four traffic classes	Capturing or classifying real Internet packets
One shared bottleneck link with configurable capacity	Multiple routers, hosts, or network paths
FIFO and SPQ queueing behavior	Real routing protocols, ECMP, or multipath load balancing
Finite buffer and tail-drop behavior	TCP congestion control, retransmissions, and transport recovery
Simulated delay, loss, throughput, and utilization	Claims about actual campus/Internet network performance

WHY THIS BOUNDARY

Keeping one link and two policies makes the project achievable while still demonstrating congestion, classification, scheduling, packet loss, and measurable QoS trade-offs.

5. System architecture

1	2	3	4	5
Traffic generator	**Classifier + priority label**	**FIFO or SPQ queues**	**Bottleneck link**	**Metrics + dashboard**

Generate → classify → queue → schedule → transmit → record → compare

6. Traffic classes

Class	Priority	Behavior represented	Reason for priority
Voice	4 — highest	Small, regular packets	Interactive voice is sensitive to delay and delay variation.
Video	3 — high	Medium-size, ongoing/bursty packets	Interactive or live video benefits from timely delivery.
Web	2 — medium	Bursty request/response traffic	Some extra waiting is usually tolerable.
Bulk file transfer	1 — lowest	Sustained background traffic	Usually more tolerant of delay than interactive media.

Priority is a project policy choice, not a universal rule for every application. The traffic labels are synthetic inputs; the classifier does not inspect real packet headers or infer applications.

7. Minimum packet record

Every generated packet must have a stable identifier and, at minimum, these fields:

- Packet ID; traffic class; priority; payload size in bytes; generation/arrival time; queue-entry time (if admitted); transmission-start time; transmission-completion/delivery time (if delivered); final status; and drop reason (if dropped).

8. Packet lifecycle

1. A traffic generator creates a packet and assigns its class, size, and arrival time.
2. The classifier maps the class to its configured priority.
3. The packet is admitted to the queue if the shared buffer has room; otherwise it is marked dropped due to buffer overflow.
4. When the link becomes free, the selected scheduler chooses the next waiting packet.
5. The packet occupies the link for its transmission time; after that, the configured propagation delay is added.
6. The simulator records packet status and timestamps; the metrics layer aggregates results by class and policy.

9. The two scheduling policies

9.1 FIFO baseline

FIFO means First In, First Out. A single queue serves packets in order of arrival. Packet class and priority do not affect service order.

9.2 Strict Priority Queuing (SPQ)

SPQ maintains one queue per traffic class. Whenever the link is ready to serve another packet, it selects the highest-priority non-empty queue: voice, then video, then web, then file transfer. FIFO order is preserved within each class queue.

> **NON-PREEMPTIVE SERVICE**
>
> SPQ does not interrupt a packet that has already started transmission. Priority applies when selecting the next packet after the link becomes free.

9.3 Expected trade-off

Under congestion, SPQ should generally reduce waiting time for high-priority packets compared with FIFO, but it may increase the waiting time or reduce the throughput of lower-priority traffic. Sustained high-priority arrivals can starve lower-priority queues. Since the model uses a shared buffer with tail drop, a new high-priority packet can still be dropped if the buffer is already full of lower-priority packets. This limitation must be acknowledged.

10. Network and buffer model

Parameter	Required behavior
Link capacity	One configurable transmission rate, expressed in Mbps.
Transmission time	Packet size in bits divided by link rate in bits per second.
Propagation delay	One fixed, configurable one-way delay applied to each successfully transmitted packet.
Buffer	A finite shared limit measured in waiting packets (excluding the packet in service); the same limit applies to FIFO and SPQ.
Drop rule	Tail drop: if the shared buffer is full at packet arrival, the arriving packet is dropped.
Service	One packet at a time on the bottleneck link; no packet preemption.
Completion	After the offered-arrival phase ends, drain all admitted packets before finalizing metrics.
Loss model	The only modeled packet loss is buffer-overflow drop; no random link errors or downstream loss.

11. Reproducibility rule

The traffic workload must be generated once and then replayed unchanged for FIFO and SPQ. The same packet IDs, classes, sizes, arrival times, link capacity, buffer limit, propagation delay, and measurement window must be used in both runs. A fixed random seed (or saved workload) must permit repeatable results.

12. Performance metrics and exact definitions

Metric	Project definition
Generated packets	Total number of packets created during the offered-arrival phase.
Delivered packets	Packets that complete transmission and the modeled propagation delay.
Dropped packets	Packets rejected because the shared queue buffer is full.
Packet-drop rate	Dropped packets ÷ generated packets × 100%.
Queueing delay	Transmission start time − queue-entry time, for an admitted packet.
End-to-end simulated delay	Delivery time − generation time. In this model it equals queueing delay + transmission time + fixed propagation delay.
Transmission time	Packet size in bits ÷ link rate in bits per second.
Delay variation estimate	For each class, the mean absolute difference between end-to-end delays of successive delivered packets ordered by generation time. Label it “delay variation estimate” or “jitter estimate”; do not claim it is a complete implementation of an Internet standard.
Run-average throughput	Delivered payload bits ÷ total simulated elapsed time, including the queue-drain phase, converted to Mbps. Report overall and per class; show the measured run duration.
Link utilization	Total simulated time the link is transmitting ÷ total measured simulation elapsed time × 100%.
Average queue occupancy	Time-weighted average number of packets waiting in the buffer.
Maximum queue occupancy	Largest observed number of waiting packets during the run.
Delivery ratio	Delivered packets ÷ generated packets × 100%.

13. Measurement and interpretation rules

- All delays are simulated values, not wall-clock measurements from a real network.
- Dropped packets have no delivery delay and must not be included in average delay or delay variation calculations.
- Define the run interval as simulation start through delivery of the last admitted packet after arrivals stop. Apply the same rule to both policies and show the resulting run duration.
- Display both aggregate and per-class metrics. Aggregate averages alone can conceal the benefit or cost of prioritization.
- Use the phrase “delay variation estimate” for the project’s simplified absolute-difference statistic. Formal IP packet delay variation metrics are defined with their own measurement parameters.
- A QoS run is not required to improve every metric or every class. Results are evidence to analyze, not a predetermined claim.

COUNT CONSISTENCY

After the simulation has drained all admitted packets, generated packets must equal delivered packets plus dropped packets. The dashboard and CSV must use the same counts.

14. Dashboard specification

The dashboard is the presentation layer for the simulation. It must make the current scenario, scheduling policy, metrics, and trade-offs clear to a viewer who has not read the code.

14.1 Controls

Control	Required behavior
Link capacity	Choose the bottleneck rate in Mbps.
Traffic-load profile	Choose Low Load, Near Capacity, Congested, or Heavy Congestion.
Buffer capacity	Set the maximum number of packets in the shared buffer.
Propagation delay	Set a fixed one-way delay in milliseconds.
Workload seed	Set or display a seed so the run can be repeated.
Run comparison	Execute FIFO and SPQ using the exact same workload and settings.
Export results	Download aggregate comparison and packet-level results as CSV.

14.2 Main views

View	What it must show
Overview	Capacity, offered load, active scenario, generated/delivered/dropped packets, drop rate, average delay, throughput, and utilization.
Per-class comparison	Voice/video/web/file table with FIFO and SPQ values side by side.
Performance charts	FIFO-vs-SPQ delay, delay-variation estimate, drop rate, and throughput; queue occupancy over simulated time.
Queue/traffic view	Class-level queue occupancy or waiting-packet counts and the current configured priority order.
Packet trace	A filterable sample of packet IDs, class, arrival time, service start, delivery/status, delay, and drop reason.
Export/report	CSV downloads and a concise statement of the scenario assumptions.

14.3 Presentation quality

- Clearly label every chart axis and every unit (ms, Mbps, packets, or %).
- Use consistent class names and priority order in all views.
- Show “Simulation results” prominently; do not imply live network monitoring.
- Do not hardcode result numbers. Every metric and graph must be derived from the current run.
- Display a clear message if a metric is undefined, such as delay variation when fewer than two packets of a class are delivered.

15. Functional requirements

ID	Requirement	Acceptance-level behavior
FR-01	Traffic classes	Generate voice, video, web, and bulk file packets with identifiable class labels.
FR-02	Configurable workload	Support defined load profiles and reproducible packet arrivals/sizes.
FR-03	Classification	Map every packet to the configured class and priority.
FR-04	FIFO baseline	Serve packets in arrival order.
FR-05	SPQ policy	Serve the highest-priority non-empty queue whenever the link becomes free; retain FIFO order within a class.
FR-06	Finite buffer	Apply the same shared-buffer capacity to both policies and record tail drops.
FR-07	Link model	Use configured link rate and fixed propagation delay.
FR-08	Packet trace	Record the lifecycle timestamps and final status for every packet.
FR-09	Metrics	Calculate all metrics in Section 12 at aggregate and per-class levels.
FR-10	Fair comparison	Replay one identical workload for FIFO and SPQ.
FR-11	Dashboard	Show controls, metrics, charts, queue/traffic details, and packet trace.
FR-12	Export	Export summary and packet-level results without changing their values.
FR-13	Validation	Check count consistency, valid metric ranges, and repeatability.

16. Non-functional requirements

Requirement	Expected standard
Correctness	Scheduling order and metric formulas match the definitions in this document.
Reproducibility	Same workload and settings produce the same results.
Usability	A new user can run the default comparison and understand the charts without editing source files.
Responsiveness	Normal CBP-sized scenarios complete in a few seconds on a student laptop.
Traceability	Displayed totals reconcile with packet-level records and CSV exports.
Honest claims	The interface and report state that the model is simulated and simplified.

17. Default experiment matrix

Scenario	Offered load target	What it demonstrates
A — Low load	About 50% of link capacity	Little queueing; policies may behave similarly.
B — Near capacity	About 90–100% of link capacity	Small changes in arrivals can increase waiting time.
C — Congested	About 130% of link capacity	Queues build and service order begins to matter.
D — Heavy congestion	About 170% of link capacity	Clear competition for link service and possible buffer drops.

These load levels are scenario targets, not guaranteed exact measured rates. The final report must show the actual offered load calculated from the workload.

18. Acceptance and validation checklist

ID	Check	Pass condition
A1	FIFO ordering	With a fixed example workload, packets start transmission in arrival order.
A2	Priority ordering	When the link becomes free and multiple queues are non-empty, SPQ selects the highest-priority non-empty queue.
A3	Non-preemption	A packet already transmitting completes before SPQ selects another packet.
A4	Buffer limit	When the shared buffer is full, a newly arriving packet is marked dropped; occupancy never exceeds capacity.
A5	Same input	FIFO and SPQ receive exactly the same packet IDs, sizes, classes, arrival times, and network settings.
A6	Count reconciliation	Generated = delivered + dropped after the admitted queue has drained.
A7	Metric ranges	Drop rate and utilization remain between 0% and 100%; delays and packet counts are non-negative.
A8	Repeatability	Repeating a run with the same seed and settings produces identical outputs.
A9	Dashboard consistency	Dashboard, charts, packet trace, and exported CSV agree.
A10	Trade-off visibility	A deterministic congestion example shows that SPQ can transmit a high-priority packet before queued lower-priority packets, while preserving non-preemption.

19. Final deliverables

- Working QoS Traffic Manager simulation with FIFO and SPQ.
- Interactive dashboard with configurable scenarios and side-by-side comparisons.
- Performance charts and filterable packet-level trace.
- CSV export of aggregate and per-packet results.
- Short project report explaining the model, assumptions, experiment matrix, actual results, trade-offs, limitations, and conclusion.
- Presentation/demo sequence and answers to basic QoS viva questions.

20. Explicit exclusions

To protect the project from scope creep, these features are not part of the required version: real packet capture or deep packet inspection; a full network simulator; multiple routers or paths; dynamic routing/ECMP; live network monitoring; TCP congestion control or retransmissions; real DSCP header marking; WFQ; traffic policing/shaping; admission control; reservations; RED/WRED/CoDel; ECN; SDN; automatic path failover; or production SLA enforcement.

21. Correct conclusion to demonstrate

PROJECT CONCLUSION TEMPLATE

"We compared FIFO and Strict Priority Queuing using the same synthetic traffic and a finite-capacity bottleneck model. The recorded results show how scheduling changes delay, delay variation, packet loss, throughput, and queue occupancy by traffic class under different load levels. QoS prioritizes service; it does not increase the physical link capacity, and prioritizing one class may reduce service quality for another."

Appendix A. Core terms for the report and viva

Term	Meaning in this project
QoS	Network mechanisms that differentiate service for traffic with different performance needs.
Packet classifier	Maps each generated packet to its known traffic class; not an AI model.
Priority	Configured service rank used by SPQ to choose the next packet.
FIFO	First In, First Out: service in packet arrival order.
SPQ	Strict Priority Queuing: always serve the highest-priority non-empty queue when service can begin.
Congestion	Offered traffic exceeds the link's transmission capacity for a period.
Queueing delay	Time spent waiting before transmission begins.
Throughput	Successfully delivered payload bits per measured simulation second.
Tail drop	Drop the packet that arrives when the applicable buffer is full.
Starvation	A low-priority queue can wait for a long time if higher-priority packets keep arriving.

Appendix B. References

The project uses simplified concepts consistent with established networking terminology. These sources explain the broader architecture and the definitions behind performance metrics; the project itself implements only the subset stated in this specification.

[1] IETF RFC 2475 — An Architecture for Differentiated Services. https://www.rfc-editor.org/rfc/rfc2475.html
Architecture; classification, marking, metering, shaping, and policing.

[2] IETF RFC 4594 — Configuration Guidelines for DiffServ Service Classes. https://www.rfc-editor.org/rfc/rfc4594.html
Traffic service classes and application-oriented service requirements.

[3] IETF RFC 7679 — A One-Way Delay Metric for IP Performance Metrics. https://www.rfc-editor.org/rfc/rfc7679.html
One-way delay measurement concepts.

[4] IETF RFC 3393 — IP Packet Delay Variation Metric for IP Performance Metrics.
https://www.rfc-editor.org/rfc/rfc3393.html
Packet delay variation terminology and measurement considerations.

[5] IETF RFC 7680 — A One-Way Loss Metric for IP Performance Metrics. https://www.rfc-editor.org/rfc/rfc7680.html
One-way packet loss and loss-ratio concepts.

FINAL SCOPE LOCK

The required project is one bottleneck link, four synthetic classes, one shared finite buffer, FIFO versus SPQ, reproducible experiments, per-class metrics, and a polished dashboard. Any feature outside that list is optional and must not delay the required deliverables.